Undefined Thesis Paper Project

A Thesis
Presented to
The Division of Mathematical and Natural Sciences
Reed College

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Arts

Riley Wood

May 2026

Approved for the Division
(TBD)

Greg Anderson

Angélica Osorno

Contents

1 Introduction to Formal Verification **1**
1.1 The Setting 1
1.1.1 The Abstraction Function 3
1.1.2 Abstract Domains 4
1.2 The Abstract Lattice 6
1.3 I actually define things 7

2 The First **11**
2.0.1 References and Labels 11
2.0.2 Footnotes and Endnotes 11
2.1 Bibliographies 11
2.1.1 Tips for Bibliographies 12

3 Tables and Graphics **13**
3.1 Tables 13
3.2 Figures 14
3.2.1 Common Modifications 14

Conclusion **15**
4.1 More info 15

A The First Appendix **17**

B The Second Appendix, for Fun **19**

Chapter 1

Introduction to Formal Verification

1.1 The Setting

I am going for a pretty gentle introduction. Maybe too gentle? I am thinking this will be the part of my thesis my mom reads to support me and I want it to make sense. We'll maybe tackle the Galois connection later

Given that we generally write programs to perform some task, it may be desirable to know that a program does what we want it to. We are interested in demonstrating this via a proof (in the mathematical sense) that a program is "correct". Frequently testing of code is used to determine that it is "correct", but for any program that say, takes an integer as input, there will be a (countably) infinite number of test cases. No amount of testing can verify that there is not some input you have not tried yet which breaks the code. Testing code can only ever provide a proof of incorrectness, that is an example of an input where the code behaves badly.

Acquiring a proof of correctness, or a demonstration that there is no input on which our program behaves incorrectly, requires more ingenuity.

talk about Rice's theorem?

The first challenge is defining what "correctness" refers to. We shall take the notion that a program is correct if for every input, it is impossible to reach an error state.

create a notion of program state (definition here)

formal definition of correctness

With these tools in hand, let us consider the following example

```
x = false
if x :
  kill
```

decide on a simple language and how to display code in the thesis

labels for lines

Here an error will be considered any run of this program in which `kill` is reached.

fix kill not rendering properly

It is quickly evident that this program can never reach the `kill` command, but let us explore how our notion of states applies here.

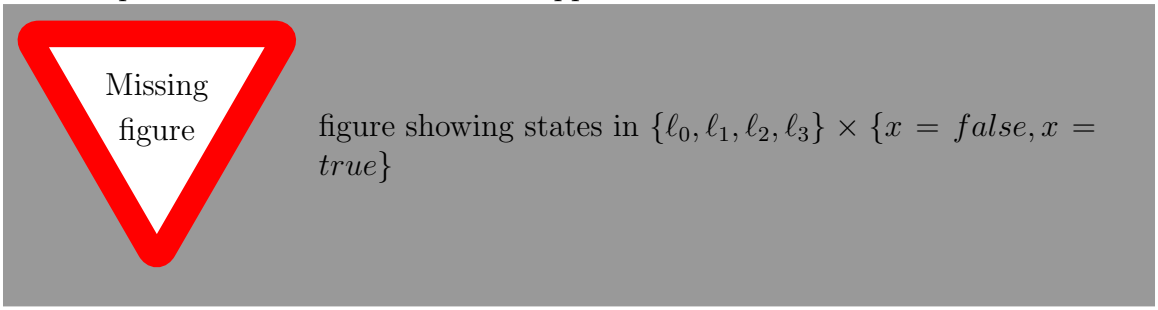


figure showing states in $\{\ell_0, \ell_1, \ell_2, \ell_3\} \times \{x = false, x = true\}$

This state diagram shows all possible program states, and we can construct the arrows between states algorithmically. For a particular state, run the next line of code, and draw an arrow to whatever state you reach. Do this for every possible state, and if there exists a path from the start state to any error state, the program fails verification.
This is the basic idea underpinning Kripke structures, a graph-based approach to checking program correctness.

define Kripke structures

The major limitation of Kripke structures is demonstrated by the following program

```
Assert x >= 0
Assert y >= 0
if x+y < 0:
  kill
```

Despite it being evident that reaching `kill` is not possible, Kripke structures as we have defined them will give us some trouble. The issue comes with the construction of the state diagram for this program, algorithmically constructing a state for every one of the infinite possible starting combination (x, y) will exceed memory limitations. The problem is that the language of Kripke structures is unable to accommodate the possible starting configurations, even though it is clear to our eyes that two positives numbers could not add to a negative number.

I don't actually know if this is a real issue with Kripke structures here. I like this program as an example for sign analysis, but I may be under representing Kripke. Need to get a proper def of Kripke structures (or drop them entirely)

To navigate this, we take an abstract view of each program variable. Rather than think of variables as tied to specific values "x is 1", we can instead consider "x is positive". This is the fundamental idea of abstract interpretation, rather than tracking specific values at each point in the program flow, we track some generalization of the values.

In this example we can apply the idea of sign analysis. We can consider if it is

possible for each variable to be negative, 0, or positive. Say α is our function which takes a variable to its abstraction. At ℓ_0 in the above program, x can be 0 or positive, so we write $\alpha(x) = \{0, +\}$. This allows us to reduce our infinite possible starting configurations down to just one:

$$\ell_0,\ \alpha(x) = \{0, +\},\ \alpha(y) = \{0, +\}.$$

Now we can think about how to perform the check $x+y < 0$. Because we are thinking of x and y in terms of abstractions of their actual values, we need to define $+$ on those abstractions. Fortunately, in this example that is straightforward:

+	-	0	+
-	-	-	{-, 0, +}
0	-	0	+
+	{-, 0, +}	+	+

really this should be a figure

Now we can compute $\alpha(x) + \alpha(y)$ by plugging all the pairs of symbols in x and y into this table and collecting all of the results in a new set:

$$\alpha(x) + \alpha(y) = \{0, +\}.$$

Now we can see that at ℓ_1, $x + y$ cannot take any negative value, allowing us to conclude that `kill` is never reached.

It may feel that I have restated the obvious in describing the possible values that x and y have taken on here, but the importance of this maneuver is that an abstraction α can be performed on a variable by a computer. This enables a computer to automatically perform the reasoning work that we did when we thought "x can't be negative". In general, abstract interpretation seeks to perform this type of work: allowing a computer to more easily make generalizations about the objects in the code we are reasoning about.

1.1.1 The Abstraction Function

As we saw in the previous example, the "abstraction function" α is critical for providing additional reasoning capabilities to the computer. In general, α is thought of as taking program variables and giving us an element in some "Abstract Domain". What we hope is that the elements in our domain will in some way be an overapproximation of the actual values. In our example, this domain consisted of possible combinations of $-, 0$, and $+$, and it is easy to see how we get an overapproximation from this choice. If x could be positive, we have $\alpha(x)$ contain "+", which represents all positive numbers, surely a good approximation for any particular positive number.

need to break this up a little

To make this notion of overapproximation formal, it is first useful to introduce the opposite operation, a concretization fucntion γ. This function takes something

in the abstract domain and tells us what "real values" it could have. In our example above, we could consider

$$\gamma(\{0\}) = \{0\} \quad \text{and} \quad \gamma(\{+\}) = \{1, 2, 3, \ldots\}.$$

With γ in hand we can describe the idea of overapproximation as follows:

$$x \subseteq \gamma(\alpha(x))$$

That is, if we take x and abstract it, then the concrete values which $\alpha(x)$ could take should include the actual value of x.

The other desired effect of α is to take some operation like $+$ and give us a version which works on elements of the abstract domain. If we let c represent some statement in the program, we are interested in an abstract version of that statement $[\![c]\!]$. The idea is that given some program state, we can apply c to get to another program state. Then what we would like is that for a state s, we can abstract it to $\alpha(s)$, and have

yuck this paragraph needs work

$$c(s) \subseteq \gamma([\![c]\!](\alpha(s))).$$

In common language, if we take some program state s and abstract all of the variables in that state to get $\alpha(s)$, then applying the abstract version of c should give us an overapproximation of simply applying c to s.

I don't really like how I am approaching this part. Maybe I should write the formal introduction to these concepts before this?

1.1.2 Abstract Domains

With the work of α clarified, we move our discussion to where alpha takes our program variables to. The abstract domain is where our approximations of variables live, and the example of sign analysis uses subsets of $\{-, 0, +\}$ as our abstract domain. It is clear that this domain gives us an overapproximation that is correct, but we may run into situations where this domain overapproximates too much for us to prove any useful properties. Consider the following program

```
Assert x >=2
if x == 1:
  kill
```

In this program it is clearly impossible for `kill` to be reached, but

$$\alpha(x) = \{+\} \quad \text{so} \quad 1 \in \gamma(\alpha(x)).$$

Thus the check `x == 1` could appear true because our abstract domain thinks that 1 is a possible value of x. The problem here is that our domain is only able to record

that x is positive, so that the information $x >= 2$ is approximated by $x > 0$. This program would then fail to be verified by sign analysis.

In this case, we can resolve the issue by moving to a more expressive domain. Rather than considering only the sign of x, we can let α give us an upper and lower bound on the values of x. In this case,

$$\alpha(x) = [2, \text{inf}]$$

think a little bit about your choice of brackets here

This domain allows us to see that

$$\gamma(\alpha(x)) = \gamma([2, \text{inf}]) = \{2, 3, \ldots\}$$

which does not contain 1. This allows us to verify that `kill` is never reached.

This has demonstrated how our choice of abstract domain can change which programs will pass verification. With a more expressive domain, we get a finer overapproximation, and hence more programs will be verifiable. Coupled with this is an increased computational cost associated with more expressive domains. In the case of intervals, we are now keeping track of two bounds per variable, and computations $[\![c]\!]$ must take both bounds into account.

do I want to make and example of how $[\![+]\!]$ works here?

Thinking about the domain of intervals allows us to verify additional programs, but it is still an overapproximation, and we can construct examples which exploit this. The interval domain is unable to account for relationships between variables, as is demonstrated by the following example program.

```
Assert 0 <= x <= 1
y = x + 1
if y == x:
  kill
```

After y is assigned

once code labels are sorted out, make this more precise

, our abstracted variables are

$$\alpha(x) = [0, 1] \quad \text{and} \quad \alpha(y) = [1, 2].$$

These intervals are correct in that x and y must fall within these ranges, but we lose information about their relationship. Thus when the equality check `y == x` is conducted, we look at the two intervals and see that indeed we could have $x = 1$ and $y = 1$. This causes our program to fail verification.



figure showing x and y's relationship, with a shaded square representing where they could be according to the interval domain, and a line representing where they could actually be

The check $x == y$ will always be false, but our overapproximation is too coarse, leading to this program failing according to our abstract analysis.

This problem is characteristic of abstract interpretation. Because our goal is to overapproximate, there will always be cases where our approximations are too large, and programs will fail verification even though they are correct. Just choosing finer approximations This indicates that we should always choose the finest possible approximation in order to verify as many programs as possible. The challenge is one of efficiency, as we briefly discussed before, the interval domain comes at a slightly higher performance cost than the sign domain. In general, a finer approximation requires us to store more information, leading to worse performance. This leads us to a tradeoff at the heart of abstract interpretation: by picking a domain which more closely approximates the actual values of program variables we are able to correctly verify more programs, however we also must pay more in terms of performance.

Programs frequently want to have some kind of relationship between variables, and the inability to model this is seen as too great a limitation for the interval domain to be particularly useful. In order to capture a finer overapproximation, we can consider a number of other candidate domains, although further exploration of those domains is left to

put furthur discussion of zonotopes and polyhedra (maybe even octagons) somewhere else

1.2 The Abstract Lattice

Despite the faults of the interval domain, it is instructive to explore some further properties of abstract interpretation in the context of intervals. As an example, consider the following program

```
Assert 0 <= x <= 2
if x <= 1 :
  y = 1
else:
  y = 0
if y == 1:
  kill
```

What are the possible values of y when we check `y == 1`?

1.3 I actually define things

Definition 1.3.1. *The* concrete lattice *consists of maps from program variables to sets of possible valuations, ordered by set inclusion on each variable.*

For a programs with integer variables, this lattice will consist of maps

$$\text{Vars} \to 2$$

is this how I want to tackle program variables?

And we order $m \leq n$ if for all variables v, $m(v) \subseteq n(v)$ **AbstractInterpretationCornellNotes**.

prove this is a lattice

Now we can define

Definition 1.3.2. *A* program state $\omega = (\ell_i, l_a)$ *consists of a location* ℓ_i *in the program, and a lattice element* l_a*.*

We write

$$\omega(x) = b$$

To say that the map l_a of state ω assigns the set of values b to variable x. We write $\ell(\omega) = \ell_i$ to say that ω has the location ℓ_i. We write $l(\omega) = l_a$ to say that ω has the lattice element l_1.

dear god

We distinguish one state ω_{init}, as the initial state of the program. This state must have as its location ℓ_0, the first label in the code, and will map all variables to the set of possible valuations at the start of execution.

For some subset of our code c, we might be interested in where we can get by running c. We write $\omega[c]\nu$ to denote that we can run c starting in state ω and it will terminate in state ν. Notice that this requires $\ell(\omega)$ to be the same as the first location in c. There could also be no valid state ν, in particular if c contains some non-terminating loop.

Definition 1.3.3. *The* concrete semantics *of a command are a relation on program states where* $\omega \leq \mu$ *if there is a computation* $\omega[c]\nu$*. We get equality only if* c *is the empty program.*

Notice that this also induces a relation on lattice elements, where $l(\omega) \leq l(\mu)$ iff $\omega \leq \mu$.

We may care about Ω, the set of all program states which could ever be reached from ω_{init} by any run of the program. This set will generally be quite large, and the work of abstract interpretation is to figure out how we can make it smaller.

For a program or part of a program, we are interested in defining how it affects the program state. Say that c is some portion of a program, this could be just one

line of our program or any number of contiguous lines, including the entire program. c will have some start location ℓ_1 and (0 or more) stop locations. Consider the set of states

$$\{\omega \in \Omega \quad \text{where} \quad \ell(\omega) = \ell_1\}.$$

Each of these states will be a possible starting configuration for running c. We will write $\omega[\![c]\!]\nu$ if ω is a possible starting configuration for c, and by running c starting at ω we can land at state ν.

Notice that c could contain a loop which never terminates, so that there may be some ω for which there is no state we reach by running c. It could also be that c could contain some nondeterministic choice so that there are multiple states we could land in. For this reason, we cannot describe c as a function on states, but rather as a relation **CMUNotesOnSemantics**.

Definition 1.3.4. *The* concrete semantics *of a program c with start location ℓ_1 are a relation on Ω so that a state ω with $\ell(\omega) = \ell_1$ is related to a state ν if there exists a computation of c starting at ω which terminates in state ν. In this case we write:*

$$\omega[\![c]\!]\nu$$

to say that the semantics c of a program relates state ω to state ν.

Is this a relation as I have defined it... no I don't think so. Do I want it to be one? Not sure. Really I have given an ordering which is what we want from the lattice anyway.

In particular, if ω and ν have locations ℓ_i and ℓ_{i+1} respectively, the relation c tells us how the line of code between those locations affects the programs variables

an example of this?

```
let a ~ [0, 1]
```

The semantics to this line will relate ω to two different program states, one where $a = 0$ and one where $a = 1$.

We can then see our lattice operations will behave as follows: The meet of $\nu \sqcap \mu$ will give us the most recent state ω where computation histories reaching ν and μ diverged. The join $\nu\mu$ will give us the program state ω which can be reached from both ν and μ, and is nearest to them in computation histories.

Definition 1.3.5. *An* Abstract Domain *is something... ??*

need to fix this guy

Definition 1.3.6. *An* abstraction function α *is a map from concrete program states to abstract program states.*

This needs to include some notion of the galois connection / adjoint functor business Also need to talk about it as a map from concrete semantics to abstract semantics.

This one will need some work

Definition 1.3.7. *The* Abstract Lattice *of a program is a relation on abstract program states $\alpha(\omega)$ by the abstract semantics of the program.*

What is going on in this lattice? Meet should give us Join should give us

Definition 1.3.8. *The* interval domain *consists of intervals over $\mathbb{R} \cup \{-\infty, \infty\}$.*

For this domain, our abstraction function α sends a variable x to the range of possible values it could take, and semantics c to some notion of how the semantics could affect the endpoints of the intervals.

This needs some work. Maybe α should be included in the abstract domain def?

Definition 1.3.9. *The* Zonotope domain *consists of noise symbols n_i with each associated to an interval over $\mathbb{R} \cup \{-\infty, \infty\}$. These intervals are restricted to $[-1, 1]$, $[-1, \infty)$, or $(-\infty, 1]$.*

The expressive power of this domain lies in how α maps program variables to the domain. We let

$$\alpha(x) = a_0 + \sum_{i=1}^{d} a_i n_i$$

where d is the number of noise symbols defined (also the dimension of the space in which our noise symbols live), and each $a_i \in \mathbb{R}$. This is an affine transformation on the noise symbols, which allows us to capture linear relationships between variables.

work an example here?

Being able to easily capture linear relationships between variables allows analysis with the Zonotope domain to verify more programs than the interval domain can handle. Not all linear relationships can be handled by this model though.

work an example here

The way that we are able to capture relationships between variables is through exploiting the noise symbols. When sampling a variable, we fix a possible value or range of values for each noise symbol, this gives us a restriction on the possible values of other variables to those values that can occur under the restricted range of noise symbols.

This is a bad explaination, and probably better handled through example, but it needs to be said somewhere

Definition 1.3.10. *The* polyhedra domain

How do we want to approach this?

Definition 1.3.11. *A* Probabilistic Polyhedron *consists of*

1. *Noise symbols $(\varepsilon_1, \ldots, \varepsilon_n) = \vec{\varepsilon} \in (\mathbb{R} \cup \{-\infty, \infty\})^n$*
2. *A map M from program variables to affine forms over $\vec{\varepsilon}$*

3. *A map* Δ *from noise symbols to probability distributions over* $\mathbb{R} \cup \{-\infty, \infty\}$

4. *A set of linear constraints described by a* $m \times n$ *matrix* A *and a* $m \times 1$ *vector* z *such that* $A\vec{\varepsilon} \leq z$.

Give the concretization

Once again we also allow α to map variables via an affine transformation on the noise symbols. The key difference from the domain of probabilistic zonotopes is that we allow arbitrary constraints between noise symbols described by A and z. This means that rather than our affine map taking a "box" of possible noise symbol values and transforming them, it now takes an arbitrary polyhedron of noise symbols.

Chapter 2

The First

2.0.1 References and Labels

Sometimes you'd like to refer to a table or figure, e.g. you can see in Figure **??** that you can rotate figures . Start by labeling your figure or table with the label command (`\label{labelvariable}`) below the caption (see the chapter on graphics and tables for examples). Then when you would like to refer to the table or figure, use the ref command (`\ref{labelvariable}`). Make sure your label variables are unique; you can't have two elements named "default." Also, since the reference command only puts the figure or table number, you will have to put "Table" or "Figure" as appropriate, as seen in the following examples:

As I showed in Table **??** many factors can be assumed to follow from inheritance. Also see the Figure **??** for an illustration.

2.0.2 Footnotes and Endnotes

You might want to footnote something.[1] Be sure to leave no spaces between the word immediately preceding the footnote command and the command itself. The footnote will be in a smaller font and placed appropriately. Endnotes work in much the same way. More information can be found about both on the CUS site.

2.1 Bibliographies

Of course you will need to cite things, and you will probably accumulate an armful of sources. This is why BibLaTeX was created. For more information about BibLaTeX and bibliographies, see our CUS site (`web.reed.edu/cis/help/latex/index.html`)[2]. There are three pages on this topic: *biblatex* (which talks about using BibLaTeX, at `/latex/biblatex.html`), TODO STYLE DOCUMENTATION and *bibman* (which covers how to make and maintain a bibliography by hand, without BibLaTeX, at at `/latex/bibman.html`). The last page will not be useful unless you have only a

[1] footnote text

[2] **reedweb:2007**

few sources. There used to be APA stuff here, but we don't need it since I've fixed this with my apa-good natbib style file.

2.1.1 Tips for Bibliographies

1. Like with thesis formatting, the sooner you start compiling your bibliography for something as large as thesis, the better. Typing in source after source is mind-numbing enough; do you really want to do it for hours on end in late April? Think of it as procrastination.

2. The cite key (a citation's label) needs to be unique from the other entries.

3. When you have more than one author or editor, you need to separate each author's name by the word "and" e.g.
   `Author = {Noble, Sam and Youngberg, Jessica},`.

4. Bibliographies made using BibTeX (whether manually or using a manager) accept LaTeX markup, so you can italicize and add symbols as necessary.

5. To force capitalization in an article title or where all lowercase is generally used, bracket the capital letter in curly braces.

6. You can add a Reed Thesis citation[3] option. The best way to do this is to use the phdthesis type of citation, and use the optional "type" field to enter "Reed thesis" or "Undergraduate thesis". Here's a test of Chicago, showing the second cite in a row[4] being different. Also the second time not in a row[5] should be different. Of course in other styles they'll all look the same.

[3] **noble:2002**
[4] **noble:2002**
[5] **reedweb:2007**

Chapter 3

Tables and Graphics

3.1 Tables

The following section contains examples of tables, most of which have been commented out for brevity. (They will show up in the .tex document in red, but not at all in the .pdf). For more help in constructing a table (or anything else in this document), please see the LaTeX pages on the CUS site.

3.2 Figures

3.2.1 Common Modifications

The following figure features the more popular changes thesis students want to their figures. This information is also on the web at `web.reed.edu/cis/help/latex/graphics.html`.

Figure 3.1: Subdivision of arc segments. You can see that $p_3 = p'_6$.

Conclusion

Here's a conclusion, demonstrating the use of all that manual incrementing and table of contents adding that has to happen if you use the starred form of the chapter command. The deal is, the chapter command in LaTeX does a lot of things: it increments the chapter counter, it resets the section counter to zero, it puts the name of the chapter into the table of contents and the running headers, and probably some other stuff.

So, if you remove all that stuff because you don't like it to say "Chapter 4: Conclusion", then you have to manually add all the things LaTeX would normally do for you. Maybe someday we'll write a new chapter macro that doesn't add "Chapter X" to the beginning of every chapter title.

4.1 More info

And here's some other random info: the first paragraph after a chapter title or section head *shouldn't be* indented, because indents are to tell the reader that you're starting a new paragraph. Since that's obvious after a chapter or section title, proper typesetting doesn't add an indent there.

Appendix A

The First Appendix

Appendix B

The Second Appendix, for Fun